Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.01 Информатика и вычислительная техника

ОТЧЕТ

Лабораторная работа № 11

По дисциплине «Теория алгоритмов и структуры данных»

Вариант № 25

Выполнил: студент группы ИВТ-25-2б
Соловьев Тимофей Николаевич

Принял: Доц. Полякова О. А.

Пермь, 2026

1. Постановка задачи

```
Постановка задачи
Задача 1.
1. Создать последовательный контейнер.
2. Заполнить его элементами стандартного типа (тип указан в варианте).
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.
Задача 2.
1. Создать последовательный контейнер.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.
Задача 3
1. Создать параметризированный класс, используя в качестве контейнера
последовательный контейнер.
2. Заполнить его элементами.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде методов параметризированного
класса.
Задача 4
1. Создать адаптер контейнера.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.
Задача 5
1. Создать параметризированный класс, используя в качестве контейнера адаптер
контейнера.
2. Заполнить его элементами.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде методов параметризированного
класса. Задача 1
1. Контейнер - список
2. Тип элементов - double
Задача 2
Тип элементов Pair (см. лабораторную работу No3).Задача 3
Параметризированный класс – Список (см. лабораторную работу No7)
Задача 4
Адаптер контейнера – очередь с приоритетами.
Задача 5
Параметризированный класс – Список
Адаптер контейнера – очередь с приоритетами.
Задание 3 Найти среднее
арифметическое и
добавить его в конец
контейнера Задание 4 Найти элементы
ключами из заданного
диапазона и удалить их
из контейнера Задание 5 К каждому элементу добавить
сумму минимального и
максимального элементов
контейнера.
```

2. Анализ

```
Программа состоит из пяти задач, демонстрирующих работу с различными контейнерами и адаптерами
стандартной библиотеки C++.

В первой задаче используется контейнер list для хранения вещественных чисел. Исходный список
инициализируется пятью значениями от полутора до пяти с половиной. Сначала вычисляется среднее
арифметическое всех элементов путем суммирования и деления на размер списка. Полученное среднее
добавляется в конец контейнера. Затем из списка удаляются все элементы, значения которых попадают в
```

интервал от двух до четырех. Для этого создается вспомогательный список элементов, подлежащих удалению, и для каждого из них вызывается метод remove. После этого находятся минимальный и максимальный элементы с помощью алгоритмов min_element и max_element. Их сумма вычисляется и прибавляется к каждому элементу исходного списка. Все промежуточные состояния выводятся на экран.

Во второй задаче используется тот же контейнер list, но для хранения объектов пользовательского типа Pair. Структура Pair является шаблонной и содержит два поля одного типа. Оператор меньше перегружен так, что сравнение выполняется по сумме полей. Оператор больше перегружен аналогичным образом. Оператор равенства сравнивает оба поля поэлементно. Оператор вывода выводит пару в круглых скобках через запятую. Исходный список содержит пять пар целых чисел. Вычисляется сумма всех элементов пар, затем среднее арифметическое как вещественное число, после чего в конец списка добавляется новая пара, у которой первое поле равно среднему значению, а второе ноль. Затем удаляются все пары, у которых сумма полей находится в диапазоне от пяти до пятнадцати включительно. После этого находятся минимальная и максимальная пары по сумме полей, вычисляется сумма их значений, и эта сумма прибавляется к каждому полю каждой пары в списке.

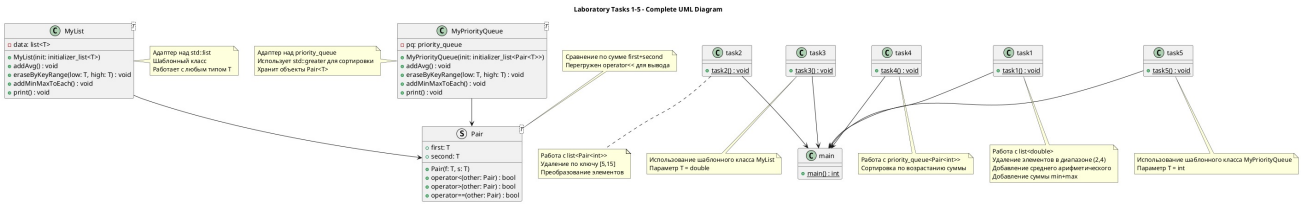
В третьей задаче создается параметризованный класс MyList, который внутри использует контейнер list. Класс имеет конструктор от списка инициализации. Метод addAvg вычисляет среднее арифметическое элементов, суммируя их с помощью оператора сложения, и добавляет результат в конец. Метод eraseByKeyRange удаляет все элементы, значения которых попадают в заданный диапазон. Метод addMinMaxToEach находит минимальный и максимальный элементы, вычисляет их сумму и прибавляет ее к каждому элементу контейнера. Метод print выводит все элементы. В демонстрации создается объект MyList для типа double, выполняются все три операции, и результаты выводятся на экран.

В четвертой задаче используется адаптер контейнера priority_queue, настроенный на хранение объектов типа Pair с использованием критерия сравнения greater, что обеспечивает извлечение элементов в порядке возрастания суммы полей. В очередь добавляются пять пар целых чисел. Для добавления среднего арифметического все элементы извлекаются из очереди в вектор, вычисляется сумма и среднее, затем создается новая пара со средним значением и нулем, которая помещается обратно в очередь. Для удаления элементов с суммой в диапазоне от шести до десяти создается временная очередь отфильтрованных элементов. После этого из оставшихся элементов находятся минимальный и максимальный по сумме, вычисляется сумма их значений, и каждое поле каждой пары увеличивается на эту сумму. Все операции сопровождаются выводом содержимого очереди.

В пятой задаче создается параметризованный класс MyPriorityQueue, который внутри использует priority_queue с элементами Pair. Конструктор принимает список инициализации пар. Метод addAvg извлекает все элементы, вычисляет сумму полей, среднее значение и добавляет новую пару. Метод eraseByKeyRange фильтрует элементы, оставляя только те, чья сумма полей вне заданного диапазона. Метод addMinMaxToEach находит минимальный и максимальный элементы, вычисляет сумму их полей и прибавляет ее к каждому полю всех пар в очереди. В демонстрации создается очередь из пяти пар целых чисел, последовательно выполняются все три операции, и после каждой выводится текущее состояние.

Основная функция последовательно вызывает все пять задач. Каждая задача работает независимо и выводит результаты своих операций. Программа демонстрирует работу со стандартными контейнерами, адаптерами, пользовательскими типами и параметризованными классами-обертками.

3. UML-диаграмма



4. Исходный код

Файл реализации (.cpp)

```
#include
#include
#include
#include
#include
#include

template
struct Pair {
    T first, second;
    Pair(T f = T(), T s = T()) : first(f), second(s) {}
    bool operator<(const Pair& other) const { return first + second < other.first + other.second; }
    bool operator>(const Pair& other) const { return first + second > other.first + other.second; }
    bool operator==(const Pair& other) const { return first == other.first && second == other.second; }
    friend std::ostream& operator<<(std::ostream& os, const Pair& p) {
```

```
        os << "(" << p.first << "," << p.second << ")"; return os;
    }
};

// Task 1
void task1() {
    std::list lst = {1.5, 2.5, 3.5, 4.5, 5.5};
    std::cout << "Task1 Initial: ";
    for (double d : lst) std::cout << d << " "; std::cout << std::endl;

    double sum = 0;
    for (double d : lst) sum += d;
    double avg = sum / lst.size();
    lst.push_back(avg);
    std::cout << "After adding avg: ";
    for (double d : lst) std::cout << d << " "; std::cout << std::endl;

    std::list toErase;
    for (double d : lst) if (d > 2.0 && d < 4.0) toErase.push_back(d);
    for (double d : toErase) lst.remove(d);
    std::cout << "After erasing (2,4): ";
    for (double d : lst) std::cout << d << " "; std::cout << std::endl;

    double minElem = *std::min_element(lst.begin(), lst.end());
    double maxElem = *std::max_element(lst.begin(), lst.end());
    double sumMinMax = minElem + maxElem;
    std::list modified;
    for (double d : lst) modified.push_back(d + sumMinMax);
    lst = modified;
    std::cout << "After adding min+max: ";
    for (double d : lst) std::cout << d << " "; std::cout << "\n\n";
}

// Task 2
void task2() {
    std::list> lst = {Pair(1,2), Pair(3,4), Pair(5,6), Pair(7,8), Pair(9,10)};
    std::cout << "Task2 Initial: ";
    for (auto& p : lst) std::cout << p << " "; std::cout << std::endl;

    int sum = 0;
    for (auto& p : lst) sum += p.first + p.second;
    double avg = sum / (double)lst.size();
    lst.push_back(Pair(avg, 0));
    std::cout << "After adding avg: ";
    for (auto& p : lst) std::cout << p << " "; std::cout << std::endl;

    int keyLow = 5, keyHigh = 15;
    lst.remove_if([keyLow, keyHigh](const Pair& p) {
        int key = p.first + p.second;
        return key >= keyLow && key <= keyHigh;
    });
    std::cout << "After erasing keys [5,15]: ";
    for (auto& p : lst) std::cout << p << " "; std::cout << std::endl;

    if (!lst.empty()) {
        auto minIt = std::min_element(lst.begin(), lst.end());
        auto maxIt = std::max_element(lst.begin(), lst.end());
        int sumMinMax = (minIt->first + minIt->second) + (maxIt->first + maxIt->second);
        std::list> modified;
        for (auto& p : lst) modified.push_back(Pair(p.first + sumMinMax, p.second + sumMinMax));
        lst = modified;
        std::cout << "After adding min+max to each element: ";
        for (auto& p : lst) std::cout << p << " "; std::cout << std::endl;
    }
    std::cout << std::endl;
}

// Task 3 Parameterized list
template
class MyList {
    std::list data;
public:
    MyList(std::initializer_list init) : data(init) {}
    void addAvg() {
        T sum = T();
        for (const auto& elem : data) sum = sum + elem;
        T avg = sum / data.size();
        data.push_back(avg);
    }
```

```
    void eraseByKeyRange(const T& low, const T& high) {
        data.remove_if([low, high](const T& elem) { return elem >= low && elem <= high; });
    }
    void addMinMaxToEach() {
        if (data.empty()) return;
        T minElem = *std::min_element(data.begin(), data.end());
        T maxElem = *std::max_element(data.begin(), data.end());
        T sumMinMax = minElem + maxElem;
        std::list modified;
        for (const auto& elem : data) modified.push_back(elem + sumMinMax);
        data = modified;
    }
    void print() const {
        for (const auto& elem : data) std::cout << elem << " ";
        std::cout << std::endl;
    }
};

void task3() {
    MyList lst = {1.1, 2.2, 3.3, 4.4, 5.5};
    std::cout << "Task3 Initial: "; lst.print();
    lst.addAvg();
    std::cout << "After adding avg: "; lst.print();
    lst.eraseByKeyRange(2.0, 4.0);
    std::cout << "After erasing [2,4]: "; lst.print();
    lst.addMinMaxToEach();
    std::cout << "After adding min+max: "; lst.print();
    std::cout << std::endl;
}

// Task 4 Priority queue adapter
void task4() {
    std::priority_queue, std::vector>, std::greater>> pq;
    std::vector> initial = {Pair(1,5), Pair(2,3), Pair(7,1), Pair(4,4), Pair(3,2)};
    for (auto& p : initial) pq.push(p);
    std::cout << "Task4 Initial priority queue (ordered by sum):\n";
    auto temp = pq;
    while (!temp.empty()) { std::cout << temp.top() << " "; temp.pop(); }
    std::cout << std::endl;

    int sum = 0;
    std::vector> vec;
    temp = pq;
    while (!temp.empty()) { vec.push_back(temp.top()); temp.pop(); }
    for (auto& p : vec) sum += p.first + p.second;
    double avg = sum / (double)vec.size();
    pq.push(Pair(avg, 0));
    std::cout << "After adding avg: ";
    temp = pq;
    while (!temp.empty()) { std::cout << temp.top() << " "; temp.pop(); }
    std::cout << std::endl;

    std::priority_queue, std::vector>, std::greater>> filtered;
    int low = 6, high = 10;
    temp = pq;
    while (!temp.empty()) {
        Pair p = temp.top(); temp.pop();
        int key = p.first + p.second;
        if (key < low || key > high) filtered.push(p);
    }
    pq = filtered;
    std::cout << "After erasing keys [6,10]: ";
    temp = pq;
    while (!temp.empty()) { std::cout << temp.top() << " "; temp.pop(); }
    std::cout << std::endl;

    if (!pq.empty()) {
        std::vector> all;
        temp = pq;
        while (!temp.empty()) { all.push_back(temp.top()); temp.pop(); }
        auto minIt = std::min_element(all.begin(), all.end());
        auto maxIt = std::max_element(all.begin(), all.end());
        int sumMinMax = (minIt->first + minIt->second) + (maxIt->first + maxIt->second);
        std::priority_queue, std::vector>, std::greater>> modified;
        for (auto& p : all) modified.push(Pair(p.first + sumMinMax, p.second + sumMinMax));
        pq = modified;
        std::cout << "After adding min+max to each: ";
        temp = pq;
        while (!temp.empty()) { std::cout << temp.top() << " "; temp.pop(); }
```

```
        std::cout << std::endl;
    }
    std::cout << std::endl;
}

// Task 5 Parameterized class with priority queue adapter
template
class MyPriorityQueue {
    std::priority_queue, std::vector>, std::greater>> pq;
public:
    MyPriorityQueue(std::initializer_list> init) {
        for (const auto& p : init) pq.push(p);
    }
    void addAvg() {
        std::vector> vec;
        auto temp = pq;
        T sum = T();
        while (!temp.empty()) { vec.push_back(temp.top()); temp.pop(); }
        for (const auto& p : vec) sum = sum + p.first + p.second;
        T avg = sum / vec.size();
        pq.push(Pair(avg, T()));
    }
    void eraseByKeyRange(const T& low, const T& high) {
        std::priority_queue, std::vector>, std::greater>> filtered;
        auto temp = pq;
        while (!temp.empty()) {
            Pair p = temp.top(); temp.pop();
            T key = p.first + p.second;
            if (key < low || key > high) filtered.push(p);
        }
        pq = filtered;
    }
    void addMinMaxToEach() {
        if (pq.empty()) return;
        std::vector> vec;
        auto temp = pq;
        while (!temp.empty()) { vec.push_back(temp.top()); temp.pop(); }
        auto minIt = std::min_element(vec.begin(), vec.end());
        auto maxIt = std::max_element(vec.begin(), vec.end());
        T sumMinMax = (minIt->first + minIt->second) + (maxIt->first + maxIt->second);
        std::priority_queue, std::vector>, std::greater>> modified;
        for (const auto& p : vec) modified.push(Pair(p.first + sumMinMax, p.second + sumMinMax));
        pq = modified;
    }
    void print() const {
        auto temp = pq;
        while (!temp.empty()) { std::cout << temp.top() << " "; temp.pop(); }
        std::cout << std::endl;
    }
};

void task5() {
    MyPriorityQueue mpq = {{1,5}, {2,3}, {7,1}, {4,4}, {3,2}};
    std::cout << "Task5 Initial: "; mpq.print();
    mpq.addAvg();
    std::cout << "After adding avg: "; mpq.print();
    mpq.eraseByKeyRange(6, 10);
    std::cout << "After erasing keys [6,10]: "; mpq.print();
    mpq.addMinMaxToEach();
    std::cout << "After adding min+max: "; mpq.print();
}

int main() {
    task1();
    task2();
    task3();
    task4();
    task5();
    return 0;
}
```

5. Результаты выполнения

```
Task1 Initial: 1.5 2.5 3.5 4.5 5.5
After adding avg: 1.5 2.5 3.5 4.5 5.5 3.5
After erasing (2,4): 1.5 4.5 5.5
```

```
After adding min+max: 8.5 11.5 12.5

Task2 Initial: (1,2) (3,4) (5,6) (7,8) (9,10)
After adding avg: (1,2) (3,4) (5,6) (7,8) (9,10) (11,0)
After erasing keys [5,15]: (1,2) (9,10)
After adding min+max to each element: (23,24) (31,32)

Task3 Initial: 1.1 2.2 3.3 4.4 5.5
After adding avg: 1.1 2.2 3.3 4.4 5.5 3.3
After erasing [2,4]: 1.1 4.4 5.5
After adding min+max: 7.7 11 12.1

Task4 Initial priority queue (ordered by sum):
(2,3) (3,2) (1,5) (4,4) (7,1)
After adding avg: (2,3) (3,2) (6,0) (1,5) (4,4) (7,1)
After erasing keys [6,10]: (2,3) (3,2)
After adding min+max to each: (12,13) (13,12)

Task5 Initial: (2,3) (3,2) (1,5) (4,4) (7,1)
After adding avg: (2,3) (3,2) (6,0) (1,5) (4,4) (7,1)
After erasing keys [6,10]: (2,3) (3,2)
After adding min+max: (12,13) (13,12)
```

6. Ответы на вопросы

```
Из каких частей состоит библиотека STL? Библиотека STL состоит из контейнеров для хранения данных,
итераторов для доступа к элементам, алгоритмов для обработки данных, функциональных объектов и адаптеров
контейнеров.

Какие типы контейнеров существуют в STL? В STL существуют последовательные контейнеры vector, list,
deque, array, forward_list, ассоциативные контейнеры set, multiset, map, multimap, неупорядоченные
ассоциативные контейнеры unordered_set, unordered_multiset, unordered_map, unordered_multimap, а также
адаптеры контейнеров stack, queue, priority_queue.

Что нужно сделать для использования контейнера STL в своей программе? Подключить соответствующий
заголовочный файл, например vector, list, и добавить using namespace std или явно использовать префикс
std.

Что представляет собой итератор? Итератор это объект, который позволяет обходить элементы контейнера
последовательно, обеспечивая единообразный доступ к элементам разных типов контейнеров.

Какие операции можно выполнять над итераторами? Над итераторами можно выполнять операции инкремента для
перехода к следующему элементу, декремента для перехода к предыдущему, разыменования для доступа к
значению, сравнения на равенство и неравенство, а также сложение и вычитание с целыми числами для
итераторов произвольного доступа.

Каким образом можно организовать цикл для перебора контейнера с использованием итератора? Использовать
цикл for с итератором от container.begin до container.end, либо применить цикл for с автоматическим
выводом типа for auto it container begin it container end it.

Какие типы итераторов существуют? Существуют итераторы ввода, итераторы вывода, однонаправленные
итераторы, двунаправленные итераторы, итераторы произвольного доступа.

Перечислить операции и методы общие для всех контейнеров. Общими являются конструкторы, деструктор,
оператор присваивания, методы begin и end, size, empty, max_size, swap.

Какие операции являются эффективными для контейнера vector Почему? Для vector эффективны операции доступа
по индексу за время O 1, добавление и удаление в конец с амортизированным временем O 1, потому что vector
хранит элементы в непрерывной области памяти.

Какие операции являются эффективными для контейнера list Почему? Для list эффективны операции вставки и
удаления в любой позиции за время O 1, потому что list реализован как двусвязный список и не требует
сдвига элементов.

Какие операции являются эффективными для контейнера deque Почему? Для deque эффективны доступ по индексу
за время O 1, добавление и удаление в начале и конце за амортизированное время O 1, потому что deque
реализован как последовательность блоков памяти.

Перечислить методы, которые поддерживает последовательный контейнер vector. Методы vector включают
assign, at, back, begin, capacity, clear, empty, end, erase, front, insert, pop_back, push_back, reserve,
resize, size, swap.

Перечислить методы, которые поддерживает последовательный контейнер list. Методы list включают assign,
back, begin, clear, empty, end, erase, front, insert, merge, pop_back, pop_front, push_back, push_front,
remove, remove_if, reverse, size, sort, splice, swap, unique.
```

Перечислить методы, которые поддерживает последовательный контейнер deque. Методы deque включают assign, at, back, begin, clear, empty, end, erase, front, insert, pop_back, pop_front, push_back, push_front, resize, size, swap.

Задан контейнер vector. Как удалить из него элементы со 2 по 5? Использовать метод erase с итераторами v.erase v.begin 2 v.begin 5.

Задан контейнер vector. Как удалить из него последний элемент? Использовать метод pop_back.

Задан контейнер list. Как удалить из него элементы со 2 по 5? Использовать метод erase с итераторами auto it l.begin advance it 2 auto it2 l.begin advance it2 5 l.erase it it2.

Задан контейнер list. Как удалить из него последний элемент? Использовать метод pop_back.

Задан контейнер deque. Как удалить из него элементы со 2 по 5? Использовать метод erase d.erase d.begin 2 d.begin 5.

Задан контейнер deque. Как удалить из него последний элемент? Использовать метод pop_back.

Написать функцию для печати последовательного контейнера с использованием итератора. template typename T void print const T container for auto it container.begin it container.end it cout it endl.

Что представляют собой адаптеры контейнеров? Адаптеры контейнеров это классы, которые предоставляют ограниченный интерфейс к базовому контейнеру, преобразуя его в структуру с определенной семантикой доступа, например stack, queue, priority_queue.

Чем отличаются друг от друга объявления stack int s и stack int list int s? stack int s использует стандартный контейнер deque, а stack int list int s использует list в качестве базового контейнера.

Перечислить методы, которые поддерживает контейнер stack. Методы stack включают push, pop, top, empty, size.

Перечислить методы, которые поддерживает контейнер queue. Методы queue включают push, pop, front, back, empty, size.

Чем отличаются друг от друга контейнеры queue и priority_queue? queue обслуживает элементы в порядке FIFO первый вошел первый вышел, а priority_queue извлекает элемент с наивысшим приоритетом согласно заданному критерию сравнения.

Задан контейнер stack. Как удалить из него элемент с заданным номером? В стеке нельзя удалить элемент с заданным номером напрямую. Нужно извлекать элементы из стека до нужного, сохраняя их в другой стек, затем пропустить удаляемый, и вернуть сохраненные элементы обратно.

Задан контейнер queue. Как удалить из него элемент с заданным номером? В очереди нельзя удалить элемент с заданным номером напрямую. Нужно извлекать элементы из очереди до нужного и помещать их обратно в конец, затем удалить нужный элемент.